

SLOpt: Serving Real-Time Inference Pipeline With Strict Latency Constraint

Zhixin Zhao , Graduate Student Member, IEEE, Yitao Hu , Guotao Yang , Ziqi Gong ,
Chen Shen , Laiping Zhao , Member, IEEE, Wenxin Li , Xiulong Liu , Senior Member, IEEE,
and Wenyu Qu , Member, IEEE

Abstract—The rise of machine learning as a service (MLaaS) has driven the demand for complex and customized real-time inference tasks, often requiring cascading multiple deep neural network (DNN) models into inference pipelines. However, these pipelines pose significant challenges due to scheduling complexity, particularly in maintaining strict latency service level objectives (SLOs). Existing systems serve pipelines with model-independent scheduling policies, which ignore the unique workload characteristics introduced by model cascading in the inference pipeline, leading to SLO violations and resource inefficiencies. In this paper, we propose that the serving system should exploit the model-cascading nature and intermodel workload dependency of the inference pipeline to ensure strict latency SLO cost-effectively. Based on this, we design and implement SLOpt, a serving system optimized for real-time inference pipelines with a three-stage codesign of workload estimation, resource provisioning, and request execution. SLOpt proposes cascade workload estimation and ahead-of-time tuning, which together address the challenge of cascade blocking and head-of-line blocking in workload estimation and resource provisioning. SLOpt further implements an adaptive batch drop policy to mitigate latency amplification issues within the pipeline. These innovations enable SLOpt to reduce the 99th percentile latency (P99 latency) by 1.4 to 2.5 times compared to the state of the arts while lowering serving costs by up to 29%. Moreover, to achieve comparable P99 latency, SLOpt requires up to 70% less cost than existing systems. Extensive evaluations on a 64-GPU cluster demonstrate SLOpt's effectiveness in meeting strict P99 latency SLOs under diverse real-world workloads.

Index Terms—Inference pipeline, real-time system, scheduling, cloud computing.

I. INTRODUCTION

THE successful application of machine learning (ML) across fields such as natural language processing, computer vision, and recommendation systems has driven the rise of Machine Learning as a Service (MLaaS), where pre-trained

ML models are offered as cloud-based services. To address the growing demand for complex and customized tasks, re-training end-to-end deep neural network (DNN) models is becoming less practical [1], [2]. Instead, a new paradigm called **model cascading** has emerged, where multiple open-access pre-trained models are cascaded into inference pipelines [3], [4], [5], [6], offering a cost-efficient and flexible alternative to training custom DNNs from scratch. Each model in the cascade performs a specific subtask, and incoming requests must pass through each model in the pipeline to generate the final result. For instance, in a traffic monitoring pipeline [7], object detection models identify vehicles in real-time video frames, and Optical Character Recognition (OCR) models further extract license plate information.

Latency Service Level Objectives (SLOs) are essential metrics for inference serving, setting the upper limit for request completion latencies. Due to workload burstiness and system scaling, end-to-end latency distributions inevitably exhibit long tail trends [8]. Consequently, existing systems [5], [9] typically use tail latency metrics, like the 99th percentile latency (P99), to assess performance. Similarly, our approach focuses on optimizing P99 latency. To meet strict P99 latency SLOs under periodic and bursty ML inference workloads, existing systems employ multiple optimization strategies: (1) reactive or predictive monitoring of workload changes for resource provisioning [4], [5], [10], (2) resource scaling to align workload demands with system capacity [3], [11], and (3) runtime adjustments such as reordering or dropping requests to maintain latency SLOs [12], [13].

Existing systems use a model-independent scheduling policy that schedules each model based on its observed workload, overlooking the model cascading feature of inference pipelines, leading to SLO violations and resource inefficiencies. The model-cascading nature of pipelines makes each model's input workload highly dependent on the output of preceding models, introducing three challenges that block existing systems from meeting strict latency SLOs: (1) *Cascade Blocking*. Delays between input workloads arriving at one model and subsequent models receiving outputs leading to the changes in the pipeline's actual input workload that cannot be immediately reflected in the observed workload of subsequent models, *i.e.*, mismatches between the actual workload and what the following models perceive, leading to cascade blocking.

Received 27 August 2024; revised 17 December 2024; accepted 4 January 2025. Date of publication 10 January 2025; date of current version 13 March 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4501000, and in part by the National Natural Science Foundation of China under Grant 62202328. (Corresponding author: Yitao Hu.)

The authors are with Tianjin Key Laboratory of Advanced Networking, Department of Intelligence and Computing, Tianjin University, Tianjin 300350, China (e-mail: yitao@tju.edu.cn).

Digital Object Identifier 10.1109/TC.2025.3528125

(2) **Head-of-Line Blocking.** During resource provisioning, model cold starts and kernel cold starts delay the scaling of model replicas, causing a mismatch between the model's estimated workload and the processing capability of its replicas. This mismatch prevents partial subsequent requests from being processed promptly, leading to head-of-line blocking.

(3) **Defect Amplification.** Under bursty serving workloads, erroneous resource provisioning leads to request queuing, which will accumulate latency and cause SLO violations. In the inference pipeline, the request accumulation becomes more severe as the workload is delivered across models, leading to defect amplification. Without properly handling these challenges, the serving system would suffer from increased latency and severe request SLO violations.

In this paper, we argue that **an effective serving system should exploit the model-cascading nature and inter-model workload dependency** to address the three challenges. Based on this, we introduce SLOpt, a serving system that enforces strict P99 latency constraints through a co-design approach integrating three stages—workload estimation, resource provisioning, and request execution. Specifically, SLOpt's distributed Cascade Estimator leverages the workload dependencies across cascaded models, providing accurate workload predictions that guide timely and appropriate resource provisioning. Building on this foundation, the AoT Tuner and State Manager eliminate kernel and model cold starts, thereby performing fast and accurate resource scaling decisions to mitigate head-of-line blocking. Finally, the Adaptive Batch Drop policy, followed after the prior two stages, mitigates defect amplification by carefully controlling request execution and dropping logic. Taken together, these three dimensions complement and reinforce one another, ensuring SLO-guaranteed, cost-effective inference serving.

This paper makes the following contributions.

- SLOpt identifies unique characteristics and challenges in serving the real-time inference pipeline, providing insights and guidance for serving system implementation. It formulates the cascade blocking issue in inference pipelines and pinpoints the kernel cold starts and model cold starts caused by autotune and model loading. Additionally, it identifies the defect amplification issue caused by suboptimal scheduling decisions.
- SLOpt leverages the unique inter-model workload dependency with an integration of three serving stages including Cascade Estimation, AoT Tuner, and Adaptive Batch Drop, to address the three challenges in serving the real-time inference pipelines. These innovations enable SLOpt to achieve state-of-the-art performance in both latency and cost-efficiency.
- SLOpt is fully implemented as a containerized system using Tensorflow Serving as inference backend with roughly 10k lines of code. Extensive evaluations on a cluster of 64 GPUs verify SLOpt's capability to minimize the P99 latency under real-world workloads [14], [15]. Specifically, compared to the state-of-the-art serving systems [4], [5] for inference pipelines, SLOpt can reduce the P99 latency by 1.4 to 2.5 times while reducing the serving cost

by up to 29.7%. Further, existing systems require 23.5% to 70.3% extra serving cost to achieve comparable P99 latency compared with SLOpt.

II. BACKGROUND AND MOTIVATION

In this section, we introduce the characteristics of inference pipeline and the workflow of serving systems (§ II-A and § II-B), then analyze the challenge in serving pipeline (§ II-C), and finally define the system requirements for guaranteeing the latency constraints for inference pipeline (§ II-D).

A. Inference Pipeline

Traditional ML workloads [12], [13], [16] typically run a single model to respond to users' inference requests, including object detection, skeleton extraction, and gaze detection. Unlike such workloads, inference pipelines cascade multiple DNN models to perform complex real-time inference tasks [5], [6]. Specifically, inference pipelines are a typical class of ML workloads with the following characteristics:

Model Cascading. In the inference pipeline, each model completes a specific subtask, and user requests must go through multiple models' inference to produce the final results. This model-cascading feature of the inference pipeline introduces additional cross-model dependencies for inference workloads, which is one of the key features that differentiate inference pipelines from traditional single-model ML workloads.

Dynamic Workload. Periodic and bursty pattern of workloads is another characteristic of inference pipelines. The inference pipeline typically responds to massive requests from different users or devices, which may exhibit periodic variations and sudden bursts. Examples include the traffic monitoring pipeline workload [17] with its diurnal pattern and the live video analytics pipeline [4], [6] with its instantaneous workload bursts during events.

Latency Constraint. Requests in the inference pipeline must satisfy strict end-to-end response latency constraints to provide real-time service, where the latency constraints are usually defined as Service Level Objectives (SLOs). However, models in the inference pipeline have distinct latency-throughput relationships, which requires the serving system to consider not only the end-to-end latency, but also the latency consumption of the request at each model.

B. Serving System

To ensure latency constraints under dynamic workloads in inference pipelines, existing serving systems [3], [4], [5], [6] employ optimization techniques such as workload prediction, resource scaling, and dynamic batching. These optimizations fall into three key runtime stages:

Workload Estimation. This stage tracks request rates for each model to estimate workload for resource provisioning. Methods are categorized into (1) reactive [3], [4], which uses observed request rates, and (2) predictive [5], [10], [11], [18], which forecasts future rates using historical data.

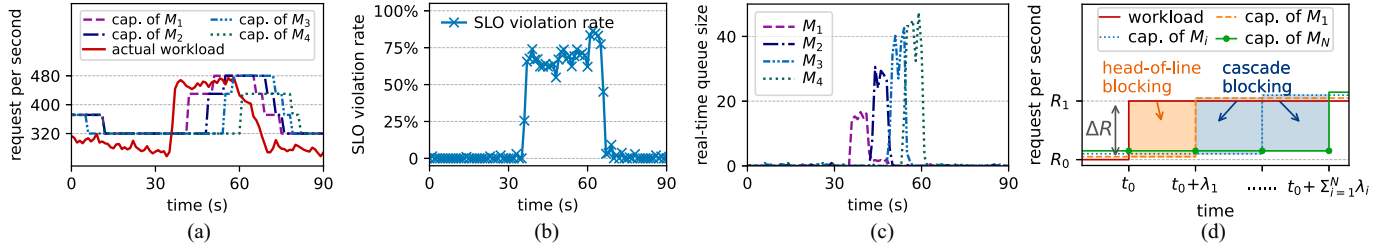


Fig. 1. (a) Actual workload and processing capability of each model. (b) End-to-end real-time SLO violation rate. (c) Real-time queue size of each model. (d) Cascade blocking and head-of-line blocking.

Resource Provisioning. Based on estimated workloads, this stage allocates and scales computation resources when the estimated workload changes [5], [11], while also adjusting runtime configurations (e.g., latency splitting) to optimize latency and throughput [3], [10].

Request Execution. This phase ensures execution latency meets SLOs and maximizes resource efficiency. Techniques include request prioritization and dropping to control latency [12], [13], as well as dynamic batching and request dispatching to enhance throughput and resource utilization of the serving system [3], [4], [19].

All three phases above are crucial and essential to guarantee the inference pipeline's SLO. However, techniques in existing serving systems usually focus on optimization for single-model ML workloads and lack analysis of the unique characteristics introduced by model cascading in the inference pipeline. Therefore, they fail to satisfy the P99 latency constraints for the inference pipeline workloads as follows.

C. Limitations of Existing Systems

Workload estimation for pipeline. In inference pipelines, requests flow sequentially through models, with each model's workload dependent on its predecessor. Due to inference duration and resource provisioning delays, workload variations at the pipeline's input are not immediately reflected downstream. This delay results in mismatches between the pipeline's actual input workloads and each model's observed workload.

Consider a pipeline with two models, M_1 and M_2 , with cold start latencies of λ_1 and λ_2 . A workload burst observed at t_0 triggers resource allocation for M_1 , but the resources become available only after λ_1 . During this period, partial requests are blocked at M_1 , delaying the reflection of the workload burst to M_2 until $t_0 + \lambda_1$ and leading to a mismatch between observed and actual workloads at M_2 . Similarly, M_2 observes the burst and allocates resources, ready at $t_0 + \lambda_1 + \lambda_2$, causing further delays and blockages. This sequential blocking, termed *cascading blocking*, arises from mismatched workload estimations.

We name the issue of requests blocking at each model sequentially due to the mismatch between observed and actual workloads as *cascading blocking*. Specifically, existing systems [3], [4], [5], [16] estimate the workload for each model solely based on the observed value, leading to the workload bursts requiring $\sum_{i=1}^N \lambda_i$ to be reflected in the last model. To make matters worse, the actual workload of the first model may have

changed after that. As shown in Figs. 1(a) and 1(b)¹, the lagging workload estimation caused by the cascading blocking results in consecutive mismatches between each model's process capability and the actual workload, which leads to a severe request SLO violation rate of up to 80%.

Observation #1: Cascade Blocking. The workload delay in the inference pipeline raises a mismatch between the observed and actual workloads, leading to inaccurate workload estimation and unacceptable P99 latency.

Resource provisioning for pipeline. The inference system adjusts resources and runtime configurations to align process capability with the estimated workload. A key challenge in resource provisioning is cold starts, which hinder efficient scaling of computational resources and runtime updates. Cold starts cause mismatches between throughput and estimated workload, leading to request accumulation and timeouts. Techniques like warmup [18] mitigate cold starts but primarily address container or function initialization, failing to address those introduced by DNN models.

To assess the cold start in model serving, we consecutively send multiple requests with varying batch sizes to models deployed on TensorFlow-Serving [20]. As shown in Fig. 2, from the execution runtime trends for SSD MobilenetV2 and ResNet50, we notice there are two levels of cold start introduced by DNN models as follows.

- **Model cold start.** It takes several steps, including deserialization, session initialization, computational graph optimization, and parameter loading, before DNN models start executing requests. These steps lead to about 3000 to 3900 milliseconds of cold start latency for the first inference of each model.
- **Kernel cold start.** CuDNN provides various algorithms and GPU kernel implementations for operators, including convolution, pooling, etc., and the optimal GPU kernel varies with batch size for specific hardware. Therefore, whenever a new batch size is used for inference, the autotune mechanism of the ML frameworks such as TensorFlow [21] and Pytorch [22], will search for the optimal kernel implementation, including algorithm and parameters, for each operator [23]. Though the autotune optimizes the inference efficiency, it introduces extra kernel cold start

¹We build a simple inference pipeline with four identical models cascaded on the baseline system [4] with the Twitter trace (Fig. 5) as input workload for this experiment.

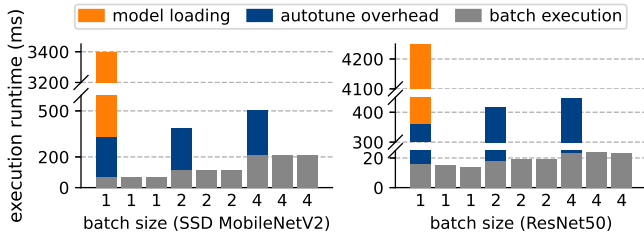


Fig. 2. Execution runtime of SSD MobilenetV2 and ResNet50.

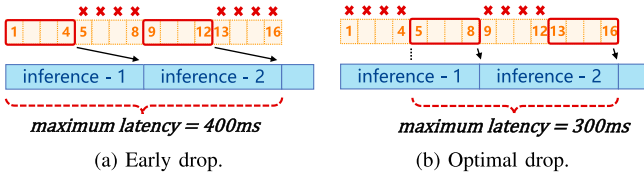


Fig. 3. Different request drop policies for workload bursts.

during batch size transitions, typically ranging from 300 to 500 milliseconds.

If model cold starts and kernel cold starts are not well handled, the serving system may suffer from severe head-of-line blocking during resource provisioning. Specifically, head-of-line blocking refers to situations where prior requests are delayed due to cold starts, preventing subsequent requests from being executed promptly, thus leading to SLO violations.

Observation #2: Head-of-Line Blocking. Model cold starts and kernel cold starts raise a mismatch between estimated workload and throughput, leading to request accumulation and latency violation.

Request execution for pipeline. Cold starts and unexpected events such as machine failures and network outages [8], [11] lead to request accumulation and latency increases. Without proper handling, the model cascading characteristic of the inference pipeline would amplify this defect.

As shown in Fig. 1(a) and Fig. 1(c), in an inference pipeline with 4 models cascaded, a spike in the workload over a 30s window leads to increasingly severe request accumulations across models. Defects such as request accumulation will be amplified by the model cascading, ultimately leading to SLO violations far beyond the theoretical lower bound (e.g., the SLO violation rate in Fig. 1(b) could be lower than 33% since the process capability of each model is about $\frac{320}{480} \approx 67\%$ of the actual workload.). Therefore, to avoid defect amplification caused by model cascading, existing systems drop or reject the execution of partial requests to ensure that the remaining ones can satisfy the SLOs [3], [4], [16]. However, the dropping policies in existing systems only consider the latency satisfaction of the current request or batch without considering whether it will affect the execution of subsequent ones. This increases unnecessary latency for all subsequent requests.

Take the early drop used in the existing system [4] as an example, which scans the request queue of each model using a sliding window with a length of batch size and executes the first batch that can satisfy the latency SLO. As shown in Fig. 3(a),

when the workload is doubled due to bursts, it executes two batches of req_{1-4} and req_{9-12} and drop req_{5-8} and req_{13-16} , since the machine is overloaded at the moment, leading to a maximum latency of 400ms. However, the maximum latency consumption at that model can be reduced to 300ms if the system drops the batches of req_{1-4} and req_{9-12} instead (Fig. 3(b)). The over-consumed latency budget by early drop will eventually lead to SLO violations due to defect amplification. Therefore, existing dropping policies for request execution cannot properly handle the inference pipeline workload, leading to higher latency consumption and sub-optimal P99.

Observation #3: Defect Amplification. The model cascading characteristic of pipeline amplifies defects, including latency budget over-consumption and request accumulation, leading to severe latency increase and SLO violation.

D. Implications

Based on the three observations above, we simplify the request SLO violations during workload bursts in the inference pipeline as Equation 1². Specifically, as shown in Fig. 1(d), assuming that there is a workload burst of ΔR at t_0 for an inference pipeline with N models cascaded, where the cold start latency of each model is $\lambda_i, i \in [1, N]$ on average. First, the observed workload and throughput of the first model experience a mismatch due to head-of-line blocking, leading to SLO violations of about $\lambda_1 \Delta R$. Then the cascade blocking at subsequent models would accumulate the SLO violations to $\sum_{i=1}^N \lambda_i \Delta R$. Finally, the inefficient request execution strategy causes more requests to accumulate, which leads to a further increase in request SLO violations caused by ΔR .

$$SLO \text{ violations} \geq \sum_{i=1}^N \lambda_i \Delta R \quad (1)$$

The limitations of existing systems for inference pipelines make it particularly desirable to have a novel, reliable serving system that is able to minimize the SLO violations and guarantee the P99 latency constraint. To achieve this goal, the inference pipeline serving system should be able to satisfy three requirements: (1) *Accurate Workload Estimation*. Due to the cascade blocking issue of the inference pipeline (**Observation #1**), it is necessary to design an accurate workload estimation strategy. (2) *Fast Resource Provisioning*. Model inference introduces the model cold start and kernel cold start (**Observation #2**), leading to severe head-of-line blocking. The serving system should mitigate these cold starts to provide fast resource provisioning. (3) *Efficient Request Execution*. Small defects are amplified by model cascades in the inference pipeline (**Observation #3**), which requires that the serving system needs to design an efficient request execution strategy to ensure that the P99 latency satisfies the real-time constraints.

²Note that partial cold start latency could be masked by preceding models' inference duration, but as shown in Fig. 15, cold start latency is an order of magnitude higher and thus omitted in Equation 1 for clarity.

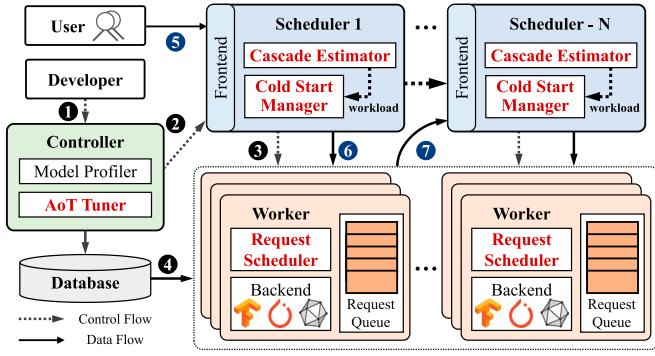


Fig. 4. SLOpt overview.

III. SYSTEM DESIGN

A. System Overview

The primary goal of SLOpt is to guarantee the P99 latency constraint for the ML inference pipeline workload. As shown in Fig. 4, developers register inference pipelines at SLOpt's controller and store the model weight and profiles in the database (1). When serving the incoming requests, the controller initializes the distributed schedulers (2), which then allocate workers (3) and load model weight to workers (4). For each incoming request, the frontend at each scheduler (5) re-directs it to the corresponding workers (6), and the worker forwards its result to the next frontend if it needs further processing (7).

SLOpt guarantees the P99 latency constraint with minimal serving cost as follows:

- It estimates the workload using distributed **Cascade Estimator**, which exploits the workload relevance between models to eliminate the workload mismatch caused by cascade blocking. It further designs a joint prediction strategy to accurately estimate the workload through an in-depth analysis of ML serving workload characteristics.
- It optimizes resource provisioning through generic model cold start optimization techniques. First, the **AoT Tuner** in the controller eliminates kernel cold starts by ahead-of-time tuning, which transforms the implicit runtime operator-kernel mapping into explicit caches. The **Cold Start Manager** then mitigates model cold starts by automatic model state transition.
- It also designs an adaptive batch dropping policy in each worker's **Request Scheduler** to ensure efficient request execution. This policy considers the latency budget of the request throughout the inference pipeline to avoid over-consumption of the latency budget due to accumulation.

B. Workload Estimation

Problems and Approaches. Accurate workload estimation is one of the essential requirements to ensure the P99 latency constraint. Existing systems can *not* derive accurate estimations for the ML inference pipeline since they make estimations for each model separately and solely based on the observed workload.

To achieve accurate workload estimation, we design a distributed Cascade Estimator in SLOpt to exploit the characteristics of the inference pipeline workload. The Cascade Estimator

divides workload estimation into two subtasks. First, it eliminates the cascade blocking issue in the inference pipeline by exploiting the workload relevance between models introduced by the pipeline's model cascading characteristic. Second, it decouples the inference workload into long-term trends, which capture the overall workload patterns over extended periods, and high-frequency fluctuations, which account for rapid, short-term changes. It then makes predictions for these two components through a joint prediction and dynamic compensation strategy. Specifically, SLOpt derives the estimation value $\tilde{R}_t$ of the workload for time t as Equation 2:

$$\tilde{R}_t = \max\{\tilde{R}_t^{pred}, \tilde{R}_t^{cas}\} + \tilde{R}_t^{fluc}, \quad (2)$$

where $\tilde{R}_t^{cas}$, $\tilde{R}_t^{pred}$ and $\tilde{R}_t^{fluc}$ denote the cascade information, long-term trend and the high-frequency fluctuation of the workload respectively.

Eliminate Cascade Blocking. As discussed in § II-A, we observe that the inputs of each model of the inference pipeline depend on the outputs of the preceding models, which introduces relevance to the workloads of each model. Furthermore, the workload between each model has a dynamic scaling relationship. For example, the workload of the license plate recognition model of the traffic monitoring pipeline [7] may be tens of times higher than that of the target recognition model during peak hours while being close to it at other periods.

The *Cascade Estimator* in SLOpt leverages workload dependencies between models to improve prediction accuracy, thereby mitigating cascade blocking. For each model in the inference pipeline, when observing a workload change, the Cascade Estimator will immediately notify its subsequent models about the change so that they can correct their observed workload based on the cascade workload from the preceding model. Specifically, for an inference pipeline, the cascade workload $\tilde{R}_t^{cas}(M_k)$ for a model M_k equals the product of two factors as Equation 3: (1) the cascade workload $\tilde{R}_t(M_{k-1})$ from its preceding model M_{k-1} and (2) M_{k-1} 's scaling factor $\tilde{S}_t(M_{k-1})$, which is the average number of outputs from M_{k-1} to M_k per input to model M_{k-1} .

$$\tilde{R}_t^{cas}(M_k) = \tilde{S}_t(M_{k-1}) \times \tilde{R}_t(M_{k-1}). \quad (3)$$

Since the $\tilde{S}_t$ is not fixed and varies across models over time [3], [5], the Cascade Estimator uses the sliding window average to derive a proper estimation for it. Specifically, the Cascade Estimator continuously collects the scaling factor of each model $\tilde{S}_t(M_k)$ and calculates the average of the latest window as an estimation of the current scaling factor.

To illustrate how it works, Table I shows the observed workload of existing systems $\tilde{R}_t^{obs}$ and the cascade workload derived by Cascade Estimator $\tilde{R}_t^{cas}$ for a given inference pipeline with three models M_1 , M_2 and M_3 cascaded, where the output of M_1 and M_2 is the input to M_2 and M_3 respectively. We assume that model M_i 's cold start latency is λ_i , and the scaling factors of M_1 and M_2 are $\tilde{S}_t(M_1) = 2$ and $\tilde{S}_t(M_2) = 4$ respectively. Initially, the estimated workload at t_0 for each model is correct (100, 200, and 800 req/sec). At t_1 , the serving system detects a 50% workload spike for M_1 , then both existing systems and

TABLE I
AN EXAMPLE OF USING CASCADE ESTIMATOR TO ELIMINATE
CASCADE BLOCKING

	Existing Method			Cascade Estimator		
	M_1	M_2	M_3	M_1	M_2	M_3
t_0	100	200	800	100	200	800
t_1	150	200	800	150	300	1200
$t_1 + \lambda_1$	150	300	800	-	-	-
$t_1 + \sum_{i=1}^2 \lambda_i$	150	300	1200	-	-	-

Cascade Estimator update the estimated workload for M_1 from 100 to 150 req/sec immediately since M_1 is the first model which has no cascade blocking issue. The newly allocated resources of M_1 will be ready at $t_1 + \lambda_1$ due to cold start latency.

With the independent estimation method, existing systems can *not* observe workload burst for M_2 and M_3 until $t_1 + \lambda_1$ and $t_1 + \sum_{i=1}^2 \lambda_i$ respectively, since subsequent models suffer from cascade blocking, leading to mismatches between observed workload and actual workload for M_2 during $[t_1, t_1 + \lambda_1]$ and for M_3 during $[t_1 + \lambda_1, t_1 + \sum_{i=1}^2 \lambda_i]$. On the contrary, with Cascade Estimator, SLOpt immediately informs M_1 's subsequent models (*i.e.*, M_2 and M_3) about the workload burst at t_1 , so M_2 and M_3 can update their workload estimation at t_1 as shown in Equation 4. Therefore, all three models are able to start resource provisioning at t_1 , which eliminates the cascade blocking for M_2 and M_3 directly.

$$\tilde{R}_{t_1}^{cas}(M_2) = \tilde{S}_{t_1}(M_1) \times \tilde{R}_{t_1}^{cas}(M_1) = 2 \times 150 = 300$$

$$\tilde{R}_{t_1}^{cas}(M_3) = \tilde{S}_{t_1}(M_2) \times \tilde{R}_{t_1}^{cas}(M_2) = 4 \times 300 = 1200 \quad (4)$$

Accurate Workload Prediction. It is not enough to use the workload at the current moment as the estimation, even if the cascading blocking is eliminated. Due to unavoidable cold starts and lagging workload change detection, it takes time for computational resources to be ready for request execution when the workload increases. Therefore, existing works [10], [11] employ predictive estimation strategies to enable timely resource provisioning. However, their prediction methods based on pre-trained models [24] are not generic because they require long-term workload-specific traces to train the model, which is unavailable before deployment.

To achieve accurate and generic workload prediction, we analyze the characteristics of ML serving workloads. As shown in Fig. 5, we find that the workload traces in existing systems [5], [10], [11] exhibit two key characteristics [13]: (1) Temporal relevance: real-world workloads typically exhibit periodic characteristics, including diurnal patterns, weekly patterns, etc. (2) High-frequency fluctuations: due to the dynamic joining and leaving of users, the workload does not change smoothly, leading to high-frequency, workload-specific and time-varying fluctuations in the trace. SLOpt exploits these characteristics to design joint prediction and dynamic compensation strategies to predict workloads as follows accurately.

First, to capture the long-term temporal relevance without being dependent on specific training data, we design a *joint prediction* method in Cascade Estimator that gradually achieves high prediction accuracy through online learning and carefully

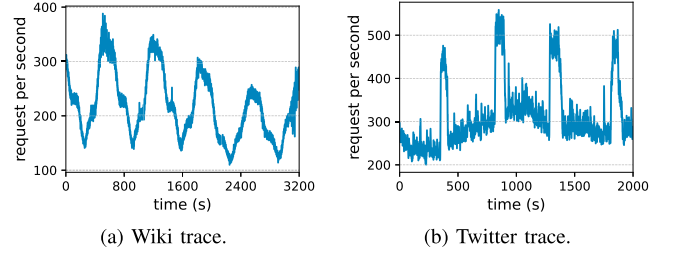


Fig. 5. Real world workload traces: Wikipedia [15] Twitter [14] access trace.

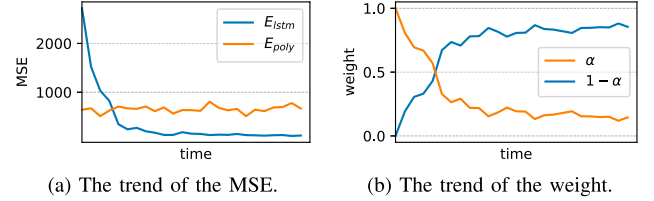


Fig. 6. The trend of MSE and weight α in joint prediction.

combines statistical-based and machine learning (ML) based prediction with dynamic weighting. The joint prediction starts with the statistical-based polynomial regression and trains a 5-layer Long Short-Term Memory (LSTM) model with collected traces using online learning. It assigns separate weights to the prediction results of the two methods, which are dynamically adjusted according to the accuracy.

At time t , the Cascade Estimator predicts the workload at $t + \lambda$ as shown in Equation 5, using polynomial regression $\tilde{R}_{t+\lambda}^{poly}$ and LSTM $\tilde{R}_{t+\lambda}^{lstm}$, weighted by α and $1 - \alpha$, respectively. Initially, $\alpha = 1.0$ since the LSTM model lacks application-specific training. After every T_u second, the Cascade Estimator re-trains the LSTM using collected workload trace and then updates the weight α based on the mean square error (MSE) of the two methods E_{poly} and E_{lstm} with $\alpha = \frac{E_{poly}}{E_{poly} + E_{lstm}}$. Fig. 6 shows the MSE and weight trends for these methods on a given workload trace. Over time, as LSTM accuracy improves, $1 - \alpha$ (weight of $\tilde{R}_{t+\lambda}^{lstm}$) surpasses α (weight of $\tilde{R}_{t+\lambda}^{poly}$), enabling a smooth transition from polynomial regression to LSTM predictions.

$$\tilde{R}_{t+\lambda}^{pred} = \alpha \tilde{R}_{t+\lambda}^{poly} + (1 - \alpha) \tilde{R}_{t+\lambda}^{lstm}. \quad (5)$$

The intuition behind the weighted combination comes from two factors: (1) During the startup phase of the serving, the statistically-based method is able to eliminate the training data dependency. (2) The statistically-based method makes it difficult to capture the time-dependent characteristics of the workloads, and thus the ML-based prediction method is eventually needed for higher accuracy.

Second, to capture the high-frequency workload fluctuation, we design a *dynamic compensation* strategy to compensate for the workload prediction $\tilde{R}_t^{pred}$. Specifically, the Cascade Estimator generates a predicted value periodically as $\{\tilde{R}_k^{pred} | k = 1, 2, \dots, K\}$, and records the actual workload value $\{\tilde{R}_k^{real} | k = 1, 2, \dots, K\}$. Then it calculates the fluctuation sequence

of workload as $\{\tilde{F}_k = \tilde{R}_k^{real} - \tilde{R}_k^{pred} | k = 1, 2, \dots, K\}$, and selects the 95th percentile of $\{\tilde{F}_k\}$ as the fluctuation compensation $\tilde{R}_t^{fluc}$. By doing so, SLOpt could identify the workload-specific fluctuations and correct the workload prediction dynamically.

Takeaway. Benefiting from the cascading estimator's cascading blocking elimination and joint prediction, SLOpt is able to make an accurate estimate of the inference pipeline workload as in Equation 2. It eliminates the observation and actual workload mismatch issue, *i.e.*, it eliminates the SLO violation in Equation 1 caused by cascade blocking and optimizes it to Equation 6.

$$SLO \text{ violations} \geq \max_{i=1}^N \lambda_i \Delta R \quad (6)$$

C. Fast Resource Provisioning

Problems and Approaches. As discussed in § II-C, cold starts raise mismatches between the estimated workload and the processing capability of the serving system, leading to delayed resource provisioning and SLO violations. To ensure the P99 latency constraint, the serving system should mitigate the cold starts, including the kernel cold starts and the model cold starts. SLOpt satisfies this requirement with its AoT Tuner and State Manager.

AoT Tuner. ML frameworks need to re-map high-level DNN operators (e.g., convolution) to specific computational algorithms (e.g., GEMM, Winograd) and optimized GPU kernels before executing a batch with a different batch size from the previous one. This runtime autotune process introduces kernel cold start to the serving system. SLOpt leverages Ahead-of-Time (AoT) Tuner to minimize the kernel cold start by caching the mapping relationship between operators and GPU kernels at different batch sizes on various hardware.

We make two observations on the autotune process. First, the overhead is non-negligible for the inference pipeline since it can reach thousands of milliseconds for different models (Fig. 15). Therefore, it is infeasible to derive the mapping relationship during online inference. Second, the mapping relationship varies with different batch sizes and hardware configurations. However, it is fixed for a given batch size on a given hardware. Therefore, it is feasible to cache the mapping relationship in advance to avoid repetitive autotune overhead.

Based on these two observations, SLOpt designs the AoT Tuner as follows. When an inference pipeline is registered, SLOpt's AoT Tuner runs the Algorithm 1 on offline mode with sample requests to search the optimal algorithm implementations and GPU kernels under different batch sizes (Line 2 to Line 5). Then, it caches the optimal mapping relationship between neural operators and GPU kernels in a shared database (Line 6). Once the pipeline is started, whenever the scheduler changes the batch size, SLOpt could load the cache of the corresponding mapping relationship directly from the shared database according to the batch size and the computation hardware (Line 10 to Line 15).

SLOpt only caches the structure information of the operator, the corresponding optimal algorithm implementation

Algorithm 1: AoT tuning and kernel launching

```

1 Input: conv_parameter, input
2 if offline_mode then
3   // offline AoT Tuning
4   algo_list = getAlgorithms(conv_parameter)
5   opt_algo = findOptAlgo(algo_list)
6   launchKernel(opt_algo, input, output)
7   // record the optimal algorithm
8   record(hardware, conv_parameter, opt_algo)
9 else
10  // online inference serving
11  opt_algo = load(hardware, conv_parameter)
12  if opt_algo == null then
13    // find usable algorithm
14    opt_algo = findAlgoHeuristic(conv_parameter)
15  launchKernel(opt_algo, input, output)
16 return output

```

(enumeration types provided by CuDNN), and GPU kernel parameters. As shown in Fig. 7(b), the cache size for several DNN models ranges only from 39 to 253 KB. Therefore, SLOpt's AoT tuner can largely reduce the kernel cold starts for up to 91% with moderate memory consumption and cache loading overhead (§ IV).

State Management. To guarantee the P99 latency constraint, model cold starts should be mitigated to avoid head-of-line blocking. Existing systems manage the DNN model replicas with only binary states (*i.e.*, the online state when the model has been loaded into GPU memory and the offline state when the model has been offloaded), which are insufficient to mitigate model cold starts. SLOpt's State Manager leverages a smart state management strategy to address this issue.

We make three key observations regarding the existing model state management and model cold starts. First, real-time serving workloads fluctuate frequently, causing DNN models to switch between online and offline, leading to repeated model load overheads. Second, DNN model parameters consume significantly less GPU memory compared to model execution [25]. Third, cloud providers offer cost-effective billing models that consider not only GPU compute time but also memory usage and idle time, allowing for differentiated pricing based on resource utilization [18], [19]. For example, prewarming instances provided by cloud services [26] allow instances to occupy a small amount of memory resources for preloading, while CPU and GPU compute resources can be elastically allocated to low-priority, preemptible tasks [27]. Based on these observations, we introduce two new states in SLOpt: *preload* and *idle*. The *preload* state allows machines to load DNN models into GPU memory in advance with low-cost containers, while the *idle* state keeps the DNN model ready for request execution with model parameters in GPU memory for a period. This strategy minimizes the probability of reloading model parameters when a new model replica is needed, thus avoiding model cold starts.

To effectively manage four model states, SLOpt needs to answer two questions: (1) what is the optimal number of model replicas, and (2) what is the state transition logic between states.

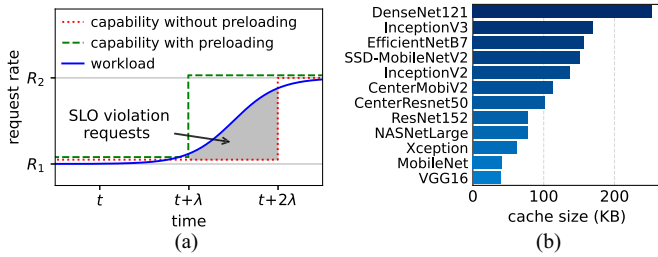


Fig. 7. (a) Avoid model cold start by preloading. (b) Cache size of DNN models using AoT tuner.

First, as shown in Fig. 7(a), to avoid model cold start during $[t + \lambda, t + 2\lambda]$, additional model replicas need to start loading at t and be ready for request execution at $t + \lambda$. Therefore, State Manager uses the same method as the Cascade Estimator to predict the number of model replicas $\tilde{N}_{t+2\lambda}$ needed for $t + 2\lambda$ at t . If the number is greater than currently available model replicas N_t , it will start preparing $\tilde{N}_{t+2\lambda} - N_t$ model replicas to preload state. Besides, the state management of State Manager follows the rule of "turn-on on-demand, turn-off delay". On the one hand, when additional model replicas are needed, the State Manager transitions replicas from *preload*, then *idle*, and finally *offline* to *online* in that order. Only the transition from *offline* to *online* involves adding new containers and incurs a model cold start. On the other hand, for any model replica (including replicas in *online* and *preload* states) that is no longer needed due to workload reduction, the State Manager sets it to *idle* state for T_w before stopping the container (*i.e.*, *offline* state), thus coping with potential workload re-increase.

By supporting multiple states, SLOpt efficiently mitigates model cold starts for the P99 latency constraint. Different from the Cascade Estimator, which handles the cascade blocking issue in Fig. 1(d) caused by unpredictable workload changes, the new states enable the serving system to make rapid reactions to predictable workload changes, minimizing the impact of model loading on latency.

$$SLO \text{ violations} \geq \max_{i=1}^N \epsilon \lambda_i \Delta R \quad (7)$$

Along with AoT Tuner and State Manager, SLOpt is able to minimize the kernel cold starts and model cold starts, thus mitigating head-of-line blocking. Consequently, SLOpt is able to further reduce the SLO violations to Equation 7 on top of Equation 6, where $\epsilon \lambda$ ($\epsilon < 1.0$) is the optimized average cold start latency.

D. Efficient Request Execution

Problems and Approaches. As discussed in § II-C, any minor defects will be amplified by the model cascading of the inference pipeline, leading to severe request accumulation and SLO violations. While the request dropping strategy employed by existing systems in the request execution stage fails to address the defect amplification issue due to latency budget over-consumption. SLOpt addresses this issue with its adaptive batch drop policy.

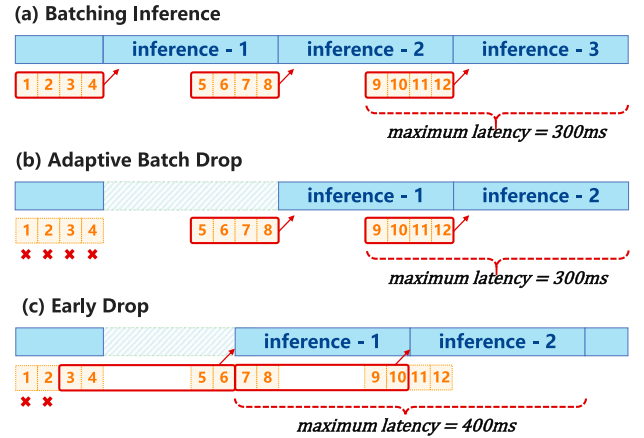


Fig. 8. Example of request dropping in a single model replica.

Adaptive Batch Drop. SLOpt's adaptive batch drop policy prevents over-consumption of latency budgets within the inference pipeline by reordering request execution and selectively dropping entire batches. Specifically, when a worker is ready to execute the next batch, the request scheduler prioritizes the most recently arrived requests. If any request in the batch is projected to exceed its allocated latency budget due to accumulated latencies, the entire batch is dropped. This strategy ensures that subsequent batches can be processed promptly without further delay. By maintaining a sufficient margin between the consumed latency and the latency SLO, the adaptive batch drop policy effectively reduces the 99th percentile latency.

Take the Fig. 3 in § II-C as an example. Existing dropping policy [4] increases the latency consumption since it executes the earliest arriving batch req_{1-4} . While SLOpt's adaptive batch drop policy drops req_{1-4} and executes req_{5-8} instead by considering the maximum latency of all requests, leading to a maximum latency of 300ms (Fig. 3(b)).

In addition, it is common that multiple model replicas serve a given model in the inference pipeline jointly. Fig. 8(a) shows the execution timeline for one of the replicas with a maximum latency of 300ms under the batch size of 4 in an ideal situation. We assume that the req_{1-2} will be timeout due to latency accumulation in preceding models. Then, SLOpt's adaptive drop policy will drop the entire batch req_{1-4} to keep the maximum latency of 300ms for the subsequent requests as shown in Fig. 8(b). However, existing systems with early drop will execute the batch req_{3-6} since it won't violate the latency SLO, which increases the maximum latency to 400 ms³. Although existing systems may drop fewer requests at a given model, SLOpt's adaptive drop policy reduces unnecessary consumption of the latency budget, thereby preserving a larger budget for subsequent models that reduce the possibility of SLO violation. This makes the system more robust to workload fluctuations, resulting in a lower P99 latency (§ IV).

$$SLO \text{ violations} \approx \max_{i=1}^N \epsilon \lambda_i \Delta R \quad (8)$$

³The system with early drop policy will not execute req_{3-4} right away, as doing so would reduce the batch size [4].

With the adaptive batch drop, SLOpt provides efficient request execution that prevents defect amplifications. Eventually, SLOpt further reduces the SLO violations to Equation 8 on top of Equation 7, thus guaranteeing the P99 latency constraint for the real-time inference pipeline.

IV. EVALUATION

A. Methodology

Implementation. We fully implement SLOpt as a distributed serving system for real-time inference pipeline with all the features described in § III with roughly 2k lines of C++ codes for the AoT Tuner and 8k lines of Python codes for the rest functionalities. Each component of SLOpt runs as a standalone container, and each worker is stateless for scalability. The control messages and user requests are transferred between components using gRPC. SLOpt supports DNN models trained by several ML frameworks, including Tensorflow [20] and PyTorch [22]. We also provide extensible APIs to support customized inference pipelines.

Testbed. We deploy SLOpt on a cluster of 16 machines with 64 NVIDIA 2080Ti GPUs. Each machine is equipped with 32 CPU cores and 128 GB RAM. The machines are connected with 1 Gbps ethernet. We virtualize the cluster into 64 containers, each assigned with one GPU. Following cloud providers [28], [29], [30], we set price for container with online model replica at \$0.5/hr and the one with preload or idle model replica at \$0.25/hr.

Workloads. For comparison, we implement two inference pipelines used in existing systems [3], [4], [5]: (1) *traffic*, which uses NASNet [31] and SSD [32] for vehicle and license plate detection with latency SLO of 600 ms; (2) *game*, which analyzes live game streams using ResNet [33], MobileNetV2 [34] and Inception-ResNet [35] with latency SLO of 400 ms. Video streams for each pipeline are collected from public datasets. The latency SLOs are determined based on profiling each model's inference latency and the specific requirements of the applications, similar to existing systems [3], [4], [5], [16]. These settings allow each model replica to perform batching, thereby fully utilizing the GPU's parallel capabilities.

Similar to existing systems [10], [11], we use two publicly available traces from Twitter [14] and Wikipedia [15] as the request sending rates of video keyframes. Each request consists of one keyframe from publicly available video streams. To match the processing capabilities of our cluster, we appropriately scale down these traces. As shown in Fig. 5, the combined workload exhibits typical model serving characteristics, including long-term periodic trends, workload-specific fluctuations, and bursts.

Baselines. We compare SLOpt against two state-of-the-art pipeline serving systems: Nexus [4], which serves the inference pipeline with a batch-aware scheduler, and InferLine [5], which manages inference pipelines with low-frequency planning and high-frequency tuning. Similar to SLOpt, these alternatives are designed to guarantee the latency objective of the inference pipeline with proper workload estimation, resource provisioning and request execution strategies.

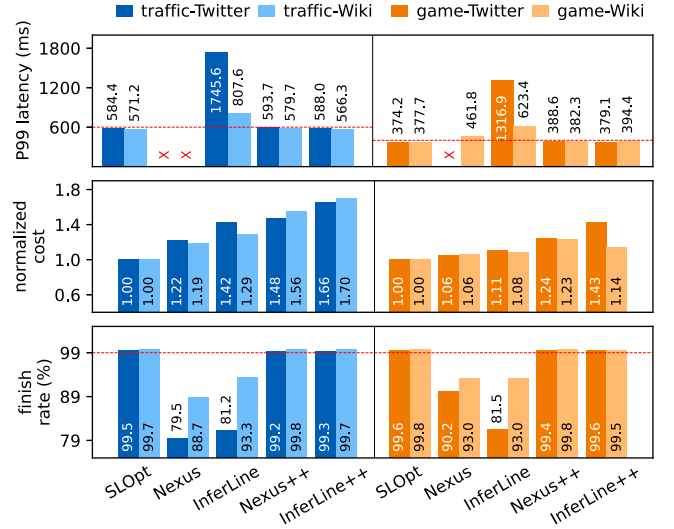


Fig. 9. P99 latency, normalized cost and finish rate of SLOpt and alternative systems. Red dashed lines mark the latency SLO for different pipelines and the 99% finish rate.

Metrics. We mainly focus on three metrics: (1) P99 latency, which is the 99th percentile latency to verify system's capability to satisfy the latency SLO. (2) Finish rate, which is the percentage of requests that have latency less than the latency SLO. Note that the request execution duration is not precisely predictable. Therefore, there could be requests with latency exceeding the latency SLO but not being dropped timely, leading to a P99 latency larger than the SLO and a finish rate lower than 99%. (3) Serving cost, which measures the amount of computation resources consumed by the serving system. The primary goal of SLOpt is to guarantee that the P99 latency is lower than the latency SLO while minimizing the serving cost.

B. Comparison Results

Fig. 9 shows the P99 latency⁴ and finish rate for SLOpt, Nexus and InferLine under two real-world traces for two inference pipelines. The P99 latency of InferLine is 1.4 to 2.5 times of SLOpt's, which can *not* satisfy the latency SLO. Nexus has no valid P99 latency in most workloads with the finish rate 6.7% to 19.8% lower than SLOpt's, leading to poor quality of service under the real-time inference pipeline workload. Meanwhile, compared to the baselines, SLOpt can reduce the serving cost by 5.3% to 29.7%. Since the performance difference comes from a combination of multiple factors, we conjecture that the advantage of SLOpt mainly comes from the following two aspects as follows. In § IV-C, we provide a detailed ablation study to quantify the importance of each design decision.

First, the workload estimation and resource provisioning strategies of existing systems can neither accurately characterize the workload nor quickly provision resources due to

⁴Note that due to request dropping, there could be more than 1% requests being dropped, leading to invalid P99 latency denoted as ×.

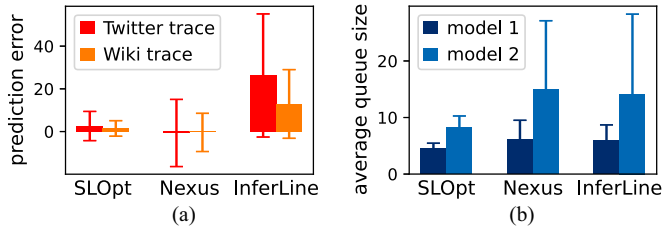


Fig. 10. (a) Prediction error for Twitter and Wikipedia trace. (b) Average queue size of each model in *traffic-Wiki* workload.

cascading blocking and head-of-line blocking. Nexus uses a reactive method for workload estimation and resource provisioning, which estimates the workload and makes resource scaling decisions solely based on the observed workload. Therefore, it achieves a smaller estimation error between the observed workload and estimated workload as shown in Fig. 10(a). However, it can *not* make proper estimation since the observed workload is not the actual workload due to cascade blocking. It also can *not* achieve fast resource provisioning due to the model cold start and kernel cold start, leading to a large amount of request dropping during resource provisioning.

Meanwhile, InferLine uses trace envelope to capture the maximum request rate within multiple sliding windows, which can easily under- or over-estimate the workload due to time-varying fluctuation, leading to a larger estimation error as shown in Fig. 10(a). Besides, the over-estimated workload leads to an extra cost of 8.1% to 42.2% than SLOpt. On the contrary, SLOpt estimates the dynamic workload accurately with its Cascade Estimator, which predicts the workload with the joint prediction and fluctuation compensation, alongside eliminating the workload mismatch caused by cascade blocking for inference pipelines. The advantage of SLOpt over Nexus and InferLine is even larger for traces with a larger fluctuation (*e.g.*, the Twitter trace).

Second, the request execution strategy of existing systems cannot efficiently deal with request accumulation, leading to severe SLO violations. Specifically, Nexus leverages early drop to drop requests that violate the latency objective as discussed in § II-C. However, Nexus ignores the dropping decisions' impact on subsequent requests and lacks proper mechanisms to mitigate the request accumulation, leading to a drop rate of larger than 1% with a finish rate of only 79.5% to 81.2% under the Twitter trace. InferLine does not drop any requests. Instead, it uses over-provisioning based on observed workload to gradually reduce the accumulated requests, leading to up to 2.5 times the P99 latency than SLOpt, which is better than Nexus. Besides, Nexus has no valid P99 latency due to request dropping and InferLine's P99 latency cannot satisfy the latency SLO. Therefore, to have a fair comparison of the capability of cost minimization, we manually increase the computation resources for the alternative systems until they have a valid P99 latency lower than the latency objective, denoted as Nexus++ and InferLine++. Fig. 9 shows that modified alternatives require a serving cost of 23.5%-55.5% and 14.4%-70.3% larger than that of SLOpt's to guarantee the P99 latency constraint, respectively.

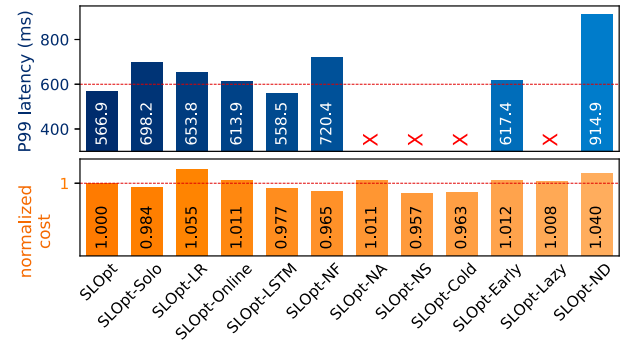


Fig. 11. P99 latency and normalized cost of SLOpt and alternative SLOpt without corresponding functionalities.

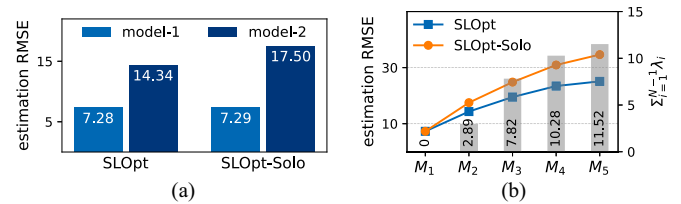


Fig. 12. (a) Estimation RMSE of SLOpt and SLOpt-Solo. (b) Estimation RMSE and cascaded cold start latency of each model in a 5-model pipeline.

C. Ablation Study

In this section, we disable each of SLOpt's features to quantify its importance in guaranteeing the P99 latency and minimizing the serving cost. For all experiments, we use the same workload as in § IV-B unless otherwise specified.

Cascade estimation. SLOpt's Cascade Estimator leverages cascade estimation to provide accurate workload estimation for the real-time inference pipeline based on workload relevance between each model. To verify its importance, we compare SLOpt against SLOpt-Solo, which disables the cascade estimation and estimates the workload of each model independently, similar to [3], [4], [5].

As shown in Fig. 11, the P99 latency for SLOpt-Solo is 23.2% higher than SLOpt's, which can *not* satisfy the latency SLO. This is because SLOpt leverages cascade estimation to quickly react to workload changes, so as to guarantee that subsequent models can correctly observe the workload changes and scale the computation resources rapidly. While SLOpt-Solo estimates the workload for each model independently without considering the model cascading characteristic of the inference pipeline, leading to delayed workload estimation (*i.e.*, cascade blocking) and resource provisioning. To verify that, Fig. 12(a) shows the RMSE for two given models using SLOpt and SLOpt-Solo, where M_1 's output is the input to M_2 . For M_1 , both SLOpt and SLOpt-Solo have the same RMSE. On the contrary, for M_2 , SLOpt-Solo's RMSE is 22.1% higher than SLOpt's. This is because it takes up to 2.89 sec for M_2 to detect the workload burst of M_1 due to cascade blocking, leading to a large mismatch between the observed workload and the actual workload. Benefiting from the cascade estimation, SLOpt scales the resources for M_1 and M_2 at the same time,

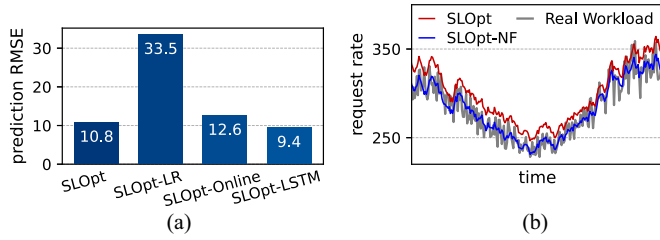


Fig. 13. (a) Workload prediction RMSE. (b) Workload estimation of SLOpt & SLOpt-NF.

which eliminates the cascade blocking and enables SLOpt to achieve a lower P99 latency with 1.6% extra serving cost than SLOpt-Solo.

Moreover, the negative impact of cascade blocking on the P99 latency is more severe when the number of models increases in the inference pipeline. Fig. 12b shows the workload estimation error for an artificial inference pipeline with five models cascaded, where it takes up to 11.5 sec for the last model to observe the workload increase due to cascade blocking. On the contrary, SLOpt reacts to the workload increase right after the workload increase, which largely reduces the RMSE of up to 38.0%.

Joint prediction. As discussed in § III-B, SLOpt uses a combination of polynomial regression and LSTM to predict the long-term workload trend for each model. To verify its importance, we compare SLOpt against: (1) SLOpt-LR, which uses polynomial regression only; (2) SLOpt-Online, which uses an LSTM model trained on a short sample workload trace and employs online learning to continuously update its weights. Specifically, request arrival intervals are periodically collected, and every 5,000 requests trigger a training process. The latest 20,000 intervals are downsampled to one-tenth of their length using a moving window average, followed by 2 training epochs to enable effective online adaptation. (3) SLOpt-LSTM, which uses an LSTM model trained on a long-term workload trace (*i.e.*, months of trace) similar to [10], [11].

Fig. 11 includes the P99 latency for the alternatives. Compared to SLOpt, SLOpt-LR and SLOpt-Online increase the P99 latency by 8.3% and 15.3%, which can *not* satisfy the latency SLO. This is because these two alternatives can *not* accurately predict the workload. To verify that, Fig. 13a shows the RMSE of SLOpt and the alternative systems. The RMSE of SLOpt-LR is 210.2% higher than SLOpt's due to prediction errors, while the one of SLOpt-Online is 16.7% higher than SLOpt's due to lack of enough training data at the beginning. Meanwhile, the P99 latency and RMSE of SLOpt-LSTM is 1.5% and 12.5% better than SLOpt's, since it is trained on a long workload trace, which is *not* available in practice. We also expanded the analysis by including additional workload prediction policies, demonstrating that LSTM achieves the lowest RMSE among all evaluated methods, as shown in Table II.

Fluctuation compensation. As discussed in § III-B, SLOpt leverages the fluctuation compensation in the Cascade Estimator to compensate for the high-frequency fluctuation. To verify its importance, we compare SLOpt against SLOpt-NF, which disables fluctuation compensation.

TABLE II
RMSE OF DIFFERENT WORKLOAD PREDICTION POLICIES

Policy	LSTM	ARIMA	Logistic R.	Linear R.	MWA
RMSE	9.43	13.8	33.1	56.3	81.2

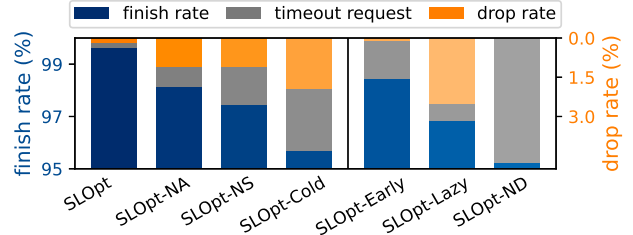


Fig. 14. Finish rate and drop rate of different solutions.

As shown in Fig. 11, the P99 latency of SLOpt-NF is 27.1% higher than SLOpt's, which can *not* satisfy the latency SLO. This is because SLOpt leverages fluctuation compensation to compensate for the prediction error rising from the workload-specific fluctuation. At the same time, SLOpt-NF has no such capability, leading to insufficient resource provisioning. To verify that, Fig. 13b shows the predicted workload of SLOpt and SLOpt-NF, alongside the actual workload as the ground truth. SLOpt's prediction can more accurately account for workload fluctuations compared to SLOpt-NF, resulting in a lower P99 latency with only a 3.65% increase in serving cost.

State Manager and AoT Tuner. Due to model cold starts and kernel cold starts, which are introduced by the model loading and autotune overhead, existing systems can *not* make timely and correct resource provisioning decisions even if the workload is estimated accurately. SLOpt's State Manager and AoT Tuner could mitigate the model loading overhead with proper state management and eliminate the autotune overhead with offline ahead-of-time tuning. To verify their importance, we compare SLOpt against: (1) SLOpt-NS, which disables State Manager to use binary model states online and offline only; (2) SLOpt-NA, which disables AoT Tuner and generates mapping from operators to GPU kernel during runtime; (3) SLOpt-Cold, which disables both State Manager and AoT Tuner.

After disabling State Manager or AoT Tuner, all the alternatives drop more than 1% of the requests, leading to invalid P99 latency in Fig. 11. Therefore, we mainly focus on the finish rate to quantitatively compare SLOpt against the alternatives. As shown in Fig. 14, SLOpt-NS, SLOpt-NA and SLOpt-Cold have a finish rate 1.5%, 2.2% and 3.9% lower than SLOpt's. This is mainly due to the model and kernel cold starts during the scheduling transition, including model loading overhead when allocating a new worker and autotune overhead when switching the batch size. The cold start generates a transient mismatch between the estimated workload and the processing capability of the serving system. Before the transient mismatch finishes, the serving system is under-provisioned, which leads to poor P99 latency due to request accumulation. To verify that, Fig. 15 shows the model and kernel cold start for different DNN models

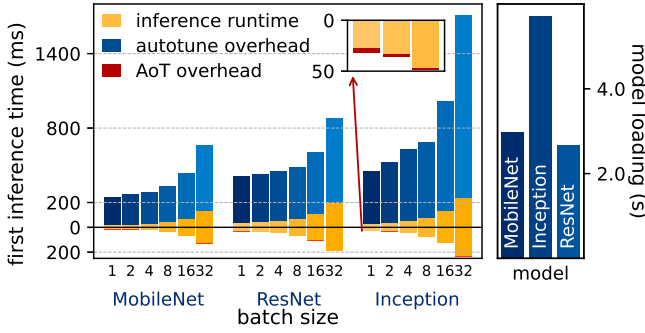


Fig. 15. (Left) First inference time with autotune and AoT tuning, where the AoT tuning reduces the autotune overhead by 97.6% to 99.8%; and (right) model loading overhead.

under various batch sizes, where the model loading overhead and autotune overhead range from 2600 to 5547 and 224 to 1477 ms. Thousands of requests will *not* be served on time during the cold start. On the contrary, SLOpt's State Manager and AoT Tuner minimize the cold starts during the scheduling transition, leading to the lowest P99 latency.

Adaptive Batch Drop. To deal with request queueing and latency violation caused by unexpected events such as network outages, SLOpt proposes Adaptive Batch Drop to drop requests for efficient request execution. To verify its importance, we compare SLOpt against: (1) SLOpt-Lazy, which uses lazy drop [16] to drop a request only when it has already missed its deadline, and determines the batch size based on the remaining latency budget; (2) SLOpt-Early, which uses early drop [4] as discussed in § II-C; (3) SLOpt-ND, which disables request dropping.

As shown in Fig. 11, the P99 latency for SLOpt-Early and SLOpt-ND is 8.9% and 61.4% higher than SLOpt's, respectively. Additionally, SLOpt-Early has a finish rate of 96.8% due to its dropping policy, leading to an invalid P99 latency. Therefore, all the three alternatives can *not* satisfy the latency SLO. To further understand it, Fig. 14 includes the corresponding finish rate and drop rate. For SLOpt-Early, though its drop rate of 0.13% is lower than SLOpt's of 0.21%, it only considers whether the current batch can be finished within the latency SLO and does *not* consider whether its execution will increase the latency of subsequent requests. Therefore, SLOpt-Early unnecessarily increases the latency budget consumption. Though its expected maximum latency is still within the latency SLO, remaining requests will be more sensitive to workload fluctuation, leading to a larger P99 latency and lower finish rate. For SLOpt-Lazy, it dynamically reduces the batch size to satisfy the latency SLO for cumulated requests. This will reduce the processing capability, leading to a higher request dropping rate of 2.53%. On the contrary, SLOpt leverages adaptive batch drop policy to drop batches that may affect the execution of subsequent requests in advance, which increases the gap between the maximum latency and the latency SLO. Therefore, SLOpt has better tolerance to workload fluctuation, leading to lower P99 latency and higher finish rate.

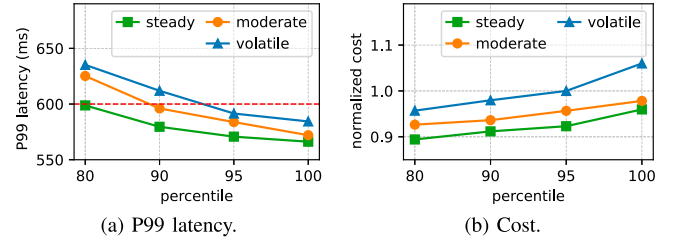


Fig. 16. Sensitivity on fluctuation compensation percentile.

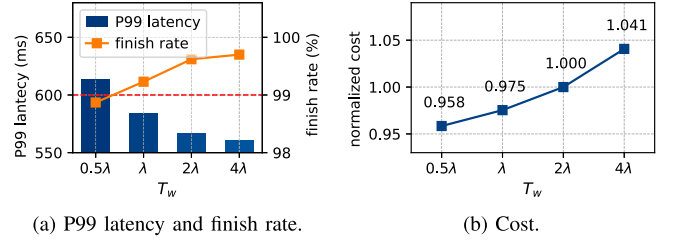


Fig. 17. Sensitivity on T_w of State Manager.

D. Sensitivity Analysis

In this section, we demonstrate that SLOpt is robust to various configurations of its hyper-parameters.

Fluctuation compensation Sensitivity. To verify the sensitivity of the 95th percentile fluctuation compensation, we filter or add noise to the Wikipedia trace to generate three workloads: steady, moderate, and volatile, which have increasing fluctuation magnitudes. Fig. 16(a) shows the effect of different fluctuation compensation percentiles on P99 latency under the three workloads, where the 95th and 100th percentiles could always guarantee P99 latency constraint. However, as shown in Fig. 16(b), the 100th percentile leads to a higher cost and is sensitive to workload fluctuations, so we choose the 95th percentile for the fluctuation compensation to compensate for the workload prediction by default.

State Manager Sensitivity. To evaluate the sensitivity for value of T_w that the State Manager maintaining the model in idle state, we vary T_w from 0.5λ to 4λ to see its impact on the P99 latency and cost, where λ is the model loading overhead. Fig. 17(a) shows the P99 latency and finish rate with different T_w , where the P99 latency is always lower than the latency SLO when $T_w > \lambda$. However, the finish rate at $T_w = \lambda$ is only 99.23%, which may violate the P99 latency constraint when the workload varies larger. Moreover, as shown in Fig. 17(b), when $T_w = 4\lambda$, the serving cost increases significantly without a comparable increase in finish rate. Therefore, we choose 2λ as the default value of T_w .

V. RELATED WORK

Existing DNN serving systems mainly focus on reducing inference latency and cost in single-model scenarios [4], [5], [7], [11]. While InferLine [5] and Cocktail [11] employ workload estimation methods, they fail to address cascade blocking across multiple models. Systems like MARK [10], [36] and Sponge [37] optimize single-model serving but do not consider the

multi-model pipeline dependencies critical to SLOpt. Recent edge-computing works target different challenges. For example, Jellyfish [38] adapts inference by switching among multiple DNN variants in response to varying network conditions and latency requirements. Context-aware approaches [39] use contextual information from edge devices to predict future data sequences and improve local scheduling. SLO-aware methods [40] dispatch tasks based on resources and workload characteristics to meet latency SLOs in multi-model co-located environments. However, these methods do not consider the inter-model workload dependencies that characterize inference pipelines, a critical challenge addressed by SLOpt. Besides, SLOpt's mitigation of model cold start is inspired by serverless-related work such as Icebreaker [18], INFless [19] and Xanadu [41], but their solutions are limited to eliminating container cold starts with warmup or predictability of DAG function calls.

There is also a large body of work to optimize the quality of service for model inference in other aspects. Gpulet [42] improves resource efficiency through GPU temporal and spatial multiplexing. The inference frameworks such as TensorRT [43] reduce inference latency and improve resource utilization through hardware-specific computational acceleration. In addition, techniques such as model quantization [44] and pruning [45] can also reduce the inference latency. As a serving system for real-time inference pipelines, SLOpt's techniques are orthogonal to these systems and can help them achieve lower latency and higher resource efficiency.

VI. CONCLUSION

In this paper, we presented SLOpt, an efficient and scalable serving system for real-time inference pipeline workloads. SLOpt identifies the cascade blocking and defect amplification issues introduced by model cascading of the inference pipeline. It further designs the Cascade Estimator, AoT Tuner, State Manager, and Adaptive Batch Drop policy for each of the three stages of the serving system to overcome the challenges introduced by real-time inference pipeline workloads. Experiment results show that SLOpt reduces the P99 latency by 1.4 to 2.5 times compared to the state-of-the-art systems while reducing the serving cost by 5.3% to 29.7%. Not limited to inference pipelines, SLOpt's design can also provide insights and guidance for fields such as serverless computing and single-model serving.

REFERENCES

- [1] M. Zaharia et al., "The shift from models to compound AI systems," 2024. [Online]. Available: <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>
- [2] M. Raghu, M. Zaharia, and E. Schmidt, "Does one large model rule them all? Predictions on the future AI ecosystem," 2023. [Online]. Available: <https://maithraraghu.com/blog/2023/does-one-model-rule-them-all/>
- [3] Y. Hu, R. Ghosh, and R. Govindan, "Scrooge: A cost-effective deep learning inference system," in *Proc. ACM Symp. Cloud Comput.*, 2021, pp. 624–638.
- [4] H. Shen et al., "Nexus: A GPU cluster engine for accelerating dnn-based video analysis," in *Proc. 27th ACM Symp. Operating Syst. Princ.*, 2019, pp. 322–337.
- [5] D. Crankshaw et al., "Inferline: Latency-aware provisioning and scaling for prediction serving pipelines," in *Proc. 11th ACM Symp. Cloud Comput.*, 2020, pp. 477–491.
- [6] S. Kim et al., "Dream: A dynamic scheduler for dynamic real-time multi-model ml workloads," in *Proc. 28th ACM Int. Conf. Archit. Supp. Program. Lang. Oper. Syst.*, vol. 4, 2023, pp. 73–86.
- [7] H. Zhang, G. Ananthanarayanan, P. Bodik, M. Philipose, P. Bahl, and M. J. Freedman, "Live video analytics at scale with approximation and delay-tolerance," in *14th USENIX Symp. Networked Syst. Des. Implementation*, 2017.
- [8] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013.
- [9] J. Zhang, S. Elnikety, S. Zarar, A. Gupta, and S. Garg, "{Model-Switching}: Dealing with fluctuating workloads in {Machine-learning-as-a-service} systems," in *Proc. 12th USENIX Workshop Hot Topics Cloud Comput. (HotCloud 20)*, 2020.
- [10] C. Zhang, M. Yu, W. Wang, and F. Yan, "Mark: Exploiting cloud services for cost-effective, slo-aware machine learning inference serving," in *USENIX Annu. Tech. Conf.*, 2019, pp. 1049–1062.
- [11] J. R. Gunasekaran, C. S. Mishra, P. Thinakaran, B. Sharma, M. T. Kandemir, and C. R. Das, "Cocktail: A multidimensional optimization for model serving in cloud," in *Proc. USENIX NSDI*, 2022, pp. 1041–1057.
- [12] A. Gujarati et al., "Serving DNNs like clockwork: Performance predictability from the bottom up," in *Proc. 14th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2020, pp. 443–462.
- [13] H. Zhang, Y. Tang, A. Khandelwal, and I. Stoica, "SHEPHERD: Serving DNNs in the wild," in *Proc. 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023, pp. 787–808.
- [14] A. A. Khaleq and I. Ra, "Cloud-based disaster management as a service: A microservice approach for hurricane Twitter data analysis," in *Proc. IEEE Global Humanitarian Technol. Conf.* Piscataway, NJ, USA: IEEE Press, 2018, pp. 1–8.
- [15] G. Urdaneta, G. Pierre, and M. Van Steen, "Wikipedia workload analysis for decentralized hosting," *Comput. Networks*, vol. 53, no. 11, pp. 1830–1845, 2009.
- [16] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," in *Proc. NSDI*, vol. 17, 2017, pp. 613–627.
- [17] I. O. de Oliveira, R. Laroca, D. Menotti, K. V. O. Fonseca, and R. Minetto, "Vehicle-Rear: A new dataset to explore feature fusion for vehicle identification using convolutional neural networks," *IEEE Access*, vol. 9, pp. 101065–101077, 2021.
- [18] R. B. Roy, T. Patel, and D. Tiwari, "Icebreaker: Warming serverless functions better with heterogeneity," in *Proc. 27th ACM Int. Conf. Architectural Support Program. Languages Operating Syst.*, 2022, pp. 753–767.
- [19] Y. Yang et al., "INFless: A native serverless system for low-latency, high-throughput inference," in *Proc. 27th ACM Int. Conf. Architect. Support Program. Languages Operating Syst.*, 2022, pp. 768–781.
- [20] C. Olston et al., "Tensorflow-serving: Flexible, high-performance ml serving," 2017, *arXiv:1712.06139*.
- [21] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *OSDI*, vol. 16, no. 2016. Savannah, GA, USA, 2016, pp. 265–283.
- [22] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," *Adv. Neur. Inf. Process. Syst.*, vol. 32, 2019.
- [23] J. Mu, M. Wang, L. Li, J. Yang, W. Lin, and W. Zhang, "A history-based auto-tuning framework for fast and high-performance dnn design on gpu," in *Proc. 57th ACM/IEEE Des. Automat. Conf. (DAC)*. Piscataway, NJ, USA: IEEE Press, 2020, pp. 1–6.
- [24] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *Int. J. Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.
- [25] A. Qiao et al., "Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning," in *OSDI*, vol. 21, 2021, pp. 1–18.
- [26] "Amazon Web Services: Set the default instance warmup," 2024. [Online]. Available: <https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-default-instance-warmup.html>
- [27] "Amazon EC2 Spot Instances," 2024. [Online]. Available: <https://aws.amazon.com/ec2/spot/>
- [28] "Amazon Web Services," 2024. [Online]. Available: <https://aws.amazon.com>
- [29] "Linux Virtual Machines Pricing," 2024. [Online]. Available: <https://azure.microsoft.com/en-us/pricing/details/virtual-machines/linux/>
- [30] "Spot Virtual Machines," 2024. [Online]. Available: <https://cloud.google.com/spot-vms/>

- [31] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2018, pp. 8697–8710.
- [32] W. Liu et al., "Ssd: Single shot multibox detector," in *Comput. Vision—ECCV 2016: 14th Eur. Conf.* Springer, 2016, pp. 21–37.
- [33] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. vision pattern Recognit.*, 2016, pp. 770–778.
- [34] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2018, pp. 4510–4520.
- [35] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," in *AAAI*, vol. 31, 2017.
- [36] C. Zhang, M. Yu, W. Wang, and F. Yan, "Enabling cost-effective, slo-aware machine learning inference serving on public cloud," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1765–1779, 2020.
- [37] K. Razavi, S. Ghafouri, M. Mühlhäuser, P. Jamshidi, and L. Wang, "Sponge: Inference serving with dynamic slos using in-place vertical scaling," in *Proc. 4th Workshop Mach. Learn. Syst.*, 2024, pp. 184–191.
- [38] V. Nigade, P. Bauszat, H. Bal, and L. Wang, "Inference serving with end-to-end latency slos over dynamic edge networks," *Real-Time Syst.*, vol. 60, pp. 1–52, 2024.
- [39] J. Chen and Y. Yang, "A context-aware approach to task scheduling for time series data prediction in mobile edge computing," *Mobile Networks Appl.*, vol. 28, no. 1, pp. 421–431, 2023.
- [40] W. Seo, S. Cha, Y. Kim, J. Huh, and J. Park, "Slo-aware inference scheduler for heterogeneous processors in edge platforms," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 18, no. 4, pp. 1–26, 2021.
- [41] N. Daw, U. Bellur, and P. Kulkarni, "Xanadu: Mitigating cascading cold starts in serverless function chain deployments," in *Proc. 21st Int. Middleware Conf.*, 2020, pp. 356–370.
- [42] S. Choi, S. Lee, Y. Kim, J. Park, Y. Kwon, and J. Huh, "Serving heterogeneous machine learning models on multi-GPU servers with spatio-temporal sharing," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2022, pp. 199–216.
- [43] "Nvidia tensorrt, an sdk for high-performance deep learning inference," 2023. [Online]. Available: <https://developer.nvidia.com/tensorrt>
- [44] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2018, pp. 2704–2713.
- [45] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE Int. Conf. Comput. Vision*, 2017, pp. 2736–2744.



Zhixin Zhao (Graduate Student Member, IEEE) received the B.E. degree in computer science and technology from the Northwest A&F University, Shaanxi, China, in 2022. He is currently working toward the Ph.D. degree in software engineering from Tianjin University, Tianjin, China. His research interests include machine learning systems and cloud computing. In particular, in machine learning systems, his research focuses on inference and deployment optimization for DNN models and LLMs.



Yitao Hu received the B.S. degree from the Department of Electrical Engineering, Shanghai Jiao Tong University, advised by Prof. Xinbing Wang and the Ph.D. degree from the Networked Systems Laboratory (NSL), University of Southern California, advised by Prof. Ramesh Govindan. He is currently an Assistant Professor with the College of Intelligence and Computing, Tianjin University, China. His research interests include networked systems at the intersection of edge/cloud computing, mobile computing, and machine learning.



Guotao Yang received the B.E. degree in software engineering in 2023 from Tianjin University, Tianjin, China, where he is currently working toward the master's degree in computer science and technology. His research interests include cloud computing and distributed systems. Specifically, his research focuses on AI training and inference systems.



Ziqi Gong received the B.E. degree in computer science and technology in 2023 from Tianjin University, China, where she is currently working toward the master's degree in computer science and technology. Her main area of research is distributed systems and resource allocation for large model inference.



Chen Shen received the B.E. degree in computer science and technology from Dalian University of Technology, Dalian, Liaoning, China, in 2023. He is currently working toward the master's degree in computer science and technology with Tianjin University, Tianjin, China. His research direction is network and cloud computing, focusing on efficient inference acceleration, and deployment of large language models.



Laiping Zhao (Member, IEEE) received the B.S. and M.S. degrees from the Dalian University of Technology, Dalian, China, in 2007 and 2009, respectively, and the Ph.D. degree from the Department of Informatics, Kyushu University, Fukuoka, Japan, in 2012. He is currently a Professor with the College of Intelligence and Computing, Tianjin University, Tianjin, China. His research interests include cloud computing and serverless computing. He has authored or co-authored more than 70 papers in international journals and conferences, including IEEE TRANSACTIONS ON CLOUD COMPUTING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, TRANSACTIONS ON SERVICE COMPUTING, ASPLOS, SC, ATC, and EuroSys.



Wenxin Li received the B.E. and Ph.D. degrees from the School of Computer Science and Technology, Dalian University of Technology, in 2012 and 2018, respectively. He is currently a Tenured Associate Professor with the College of Intelligence and Computing, Tianjin University, China. He also worked as a Research Assistant with the National University of Defense Technology, China, as a Visiting Student with the Department of Electrical and Computer Engineering, University of Toronto, Canada, as a Postdoctoral Fellow with The Hong Kong University of Science and Technology, Hong Kong, and as a Researcher with the Theory Laboratory of Huawei, Hong Kong. His research interests include datacenter networks and cloud computing. His research papers were published in many prestigious journals and conferences, including TON, TPDS, INFOCOM, ICDCS, and ICPP. He was a recipient of CCF Outstanding Doctoral Dissertation Award 2020.



Xiulong Liu (Senior Member, IEEE) received the B.E. and Ph.D. degrees from Dalian University of Technology, China, in 2010 and 2016. He is currently a Professor with the College of Intelligence and Computing, Tianjin University, China. He also worked as a Visiting Researcher with Aizu University, Japan; a Postdoctoral Fellow with The Hong Kong Polytechnic University, Hong Kong; and a Postdoctoral Fellow with the School of Computing Science, Simon Fraser University, Canada. His research interests include

wireless sensing and communication, indoor localization, and networking. His research papers were published in many prestigious journals and conferences, including IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON COMMUNICATIONS, and ICNP. He received the Best Paper Awards from ICA3PP 2014 and IEEE SYSTEMS JOURNAL 2017.



Wenyu Qu (Member, IEEE) received the bachelor's and master's degrees from the Dalian University of Technology, China, in 1994 and 1997, respectively, and the Ph.D. degree from Japan Advanced Institute of Science and Technology, Japan, in 2006. She was a Professor with Dalian Maritime University, China, from 2007 to 2015. She was an Assistant Professor with Dalian University of Technology, China, from 1997 to 2003. She is currently a Professor with the College of Intelligence and Computing, Tianjin University. She has authored over 80 technical articles in international journals and conferences. Her research interests include

cloud computing, computer networks, and information retrieval. She is on the committee board for a couple of international conferences.